

# Quantization Precision Degradation and Latency Predictability in Deterministic Local LLM Ingestion Pipelines

Yashrith Chittoor Hari Krishna

Independent Researcher

B.Tech in Computer Science and Engineering

Email: yashrithchittoor@gmail.com

## Abstract

Modern edge software architectures increasingly rely on localized, post-training quantized Large Language Models (LLMs) to programmatically generate structured data payloads, such as rigid JavaScript Object Notation (JSON) arrays, for automated backend database ingestion and reactive user interface rendering. While weight quantization significantly minimizes physical memory footprints to match localized consumer-grade hardware constraints, its systemic impact on deterministic structural syntax adherence under varied sampling entropy configurations remains critically under-explored in distributed systems research. This paper introduces a multi-dimensional empirical benchmark evaluating the structural architectural reliability across three localized open-source model scales: Llama 3.2 (3B Baseline), Llama 3.1 (8B Standard), and Qwen 2.5 (14B Hardware Stress Boundary). Each model tier was subjected to 1,200 continuous, sequential schema validation queries across a strict temperature matrix ( $\tau \in \{0.0, 0.3, 0.7, 1.0\}$ ) executed on an Apple M4 Silicon unified memory platform. Structural compliance and data-type binding were programmatically verified using an unyielding Pydantic validation framework. Our experimental results expose an explicit structural “syntax rot” inflection point: under low-entropy conditions ( $\tau \leq 0.3$ ), all model scales achieved absolute structural fidelity with a 0.0% failure rate. However, under high-entropy configurations ( $\tau \geq 0.7$ ), the highly compressed 3B parameter baseline model experienced acute architectural degradation, registering a structural syntax failure rate of up to 6.0% due to quantization rounding noise outstepping syntactic constraints. Conversely, the 8B and 14B architectures exhibited pronounced structural resilience, maintaining failure rates at or below 1.0% and 2.0% respectively. Local systems profiling revealed a clear 3x latency premium for the 14B model ( $\mu = 8.49$ s) relative to the 3B baseline ( $\mu = 2.94$ s). Crucially, long-duration tail-latency analysis showed perfectly stable mean-to- $P_{95}$  bounds across the complete continuous execution window, demonstrating that localized system-on-chip (SoC) architectures can maintain highly sustainable, predictable throughput without memory fragmentation or thermal throttling anomalies. We leverage these empirical findings to provide actionable deployment guidelines for production MLOps engineering pipelines.

## Index Terms

Large Language Models, Post-Training Quantization, Syntax Rot, Structured Outputs, Pydantic Validation, Tail Latency ( $P_{95}$ ), Hardware Performance Stability.



In modern distributed computing and microservices architecture, passing data safely between decoupled application layers relies on strict data serialization contracts, typically implemented via JavaScript Object Notation (JSON). When an asynchronous application layer invokes an artificial intelligence asset—specifically a generative Large Language Model (LLM)—to extract operational parameters, update a relational SQL database, or construct state payloads for front-end user interface rendering, the generative text output must precisely match a rigid structural schema definition. If the model introduces non-deterministic conversational prose preambles, omits a syntax-critical structural delimiter, or fails to emit a terminal closing block character, downstream serialization utilities encounter unhandled exceptions, triggering an immediate crash of the data ingestion pipeline.

To escape the financial overhead, data ownership hazards, and network latency instabilities inherent in cloud-centric API dependencies, engineering teams are aggressively shifting toward localized edge deployment paradigms [1]. To make hosting large parameters viable on localized edge computing hardware, enterprise applications rely on post-training quantization (PTQ) [5]. This compression mechanism downscales the numerical precision of an LLM’s underlying parameter weights from 16-bit floating-point decimals (FP16) down to compact 4-bit block-wise integer formats, such as Ollama’s standard Q4\_K\_M format [6]. While quantization successfully reduces physical memory footprints by up to 75%, it introduces mathematical rounding noise into the model’s token-prediction probability distribution [7]. This noise can systematically destabilize the structural formatting rules embedded within the model’s parameters.

Existing industry benchmarks primarily evaluate cognitive “model intelligence” through general knowledge trivia, abstract legal analysis, or mathematical word problems. However, these frameworks fail to evaluate a model’s architectural reliability as a production software infrastructure component [3]. As local LLMs are integrated directly into complex domain pipelines—such as Natural-Language-to-SQL utilities within highly regulated industrial and biopharmaceutical manufacturing sectors [2]—the model’s capacity to maintain absolute syntax constraints over sustained runtime windows becomes paramount. A model may exhibit exceptional abstract reasoning or semantic coherence, yet remain entirely unviable for production integration if its structural output cannot cross a rigid schema parsing boundary without a system crash.

This research addresses this gap by executing a multi-dimensional structural stress test. By driving three distinct scales of 4-bit quantized models to generate strict user-profile payloads over 1,200 continuous requests, we map the structural boundaries of what we define as Syntax Rot—the structural formatting degradation of formatted outputs driven by the interplay of model scale compression, sampling entropy ( $\tau$ ), and sustained compute duration on localized system-on-chip hardware.

The primary contributions of this paper are threefold:

- 1) We isolate the exact mathematical inflection point where quantization noise overpowers syntactic boundaries as sampling temperature escalates, demonstrating that syntax rot is an algorithmic property of sampling entropy rather than a linear failure mode.
- 2) We profile the micro-level latency distributions and tail latencies ( $P_{95}$ ) of localized edge models, establishing the precise processing premiums and jitter characteristics inherent in scaling from 3B edge parameters to 14B boundary parameters.
- 3) We confirm the systemic stability of consumer-grade system-on-chip (SoC) platforms under

prolonged compute loads, showing that unified memory environments can sustain highly predictable, flat performance timelines under enterprise conditions.

To support reproducibility and further research, our complete experimental suite, raw datasets, analysis scripts, and visualization tools are publicly available on GitHub at <https://github.com/yashrith/local-llm-structured-output-reliability>.

The remainder of this paper is organized as follows. Section 2 outlines our hardware environment, model profiles, and the Pydantic structural validation pipeline. Section 3 presents our empirical results, analyzing the syntax matrix, boxplot distributions, and SLA metrics. Section 4 details the systems implications and actionable MLOps deployment paradigms. Section 5 reviews related work in localized LLM serving, and Section 6 concludes with future research directions.

## 2 Experimental Methodology

### 2.1 Hardware and Local Compute Infrastructure Platform

All evaluation cycles were executed locally on an Apple M4 Silicon System-on-Chip (SoC) architecture configured with 24 GB of hardware unified memory and 500 GB of non-volatile solid-state storage (SSD). On the Apple M4 platform, the unified memory architecture binds the CPU cores and the 10-core GPU directly to the same fast LPDDR5X memory pool, achieving an ultra-wide memory bandwidth that drastically mitigates traditional PCIe bus data-copying bottlenecks.

However, macOS resource allocation policies aggressively enforce conservative VRAM safety thresholds, capping the maximum active allocation ceiling for dedicated graphics allocations at approximately 16 GB to 18 GB to preserve overall system responsiveness. The software orchestration environment was isolated using a Python 3.12 virtual environment operating over a localized Ollama container server engine runtime [6], accessing the hardware acceleration blocks directly through Apple’s Metal Performance Shaders (MPS) framework.

### 2.2 Model Selection Coordinates and Scale Gradient

To isolate how parameter volume and network capacity affect resilience against syntax degradation, we established a distinct three-tier scale gradient using default 4-bit quantized configurations compressed via the asymmetric block-wise Q4\_K\_M quantization standard:

- Small Baseline Tier: Llama 3.2 (3B) operating at an active idle memory footprint of  $\sim 2.0$  GB. This represents the absolute lower limit of device-optimized local parameters.
- Medium Industry Standard: Llama 3.1 (8B) operating at an active idle memory footprint of  $\sim 4.7$  GB. This tier represents the current standard for localized commercial deployments.
- Large Hardware Stress Boundary: Qwen 2.5 (14B) operating at an active idle memory footprint of  $\sim 9.0$  GB. This model tier explicitly puts significant resource pressure on our machine’s 16 GB graphics allocation budget during sustained generation cycles, serving as an ideal simulation of an enterprise platform running near maximum hardware utilization.

### 2.3 Programmatic Ingestion Design and Schema Definition

The evaluation framework subjected the models to a rigid target blueprint requiring the generation of an ingestion payload containing exactly three independent user records. To ensure the programmatic

referee engine remained completely unbiased and immune to semantic interpretation, we constructed a strict validation contract utilizing Python’s type-hinting engine via a nested Pydantic metadata class definition [4], as structured in Figure 1:

```
from typing import List
from pydantic import BaseModel, Field

class UserProfile(BaseModel):
    user_id: int = Field(description="Incremental integer starting from 1")
    username: str = Field(description="Lowercase alphanumeric string")
    email: str = Field(description="Valid email address ending in .com")
    account_active: bool = Field(description="Boolean True or False flag")
    score: float = Field(description="Decimal score between 0.0 and 100.0")

class DatabasePayload(BaseModel):
    profiles: List[UserProfile]
```

Fig. 1. Pydantic data validation blueprint and structural contract definition.

To completely eliminate conversational noise, conversational filler, or structural variations introduced by traditional Markdown syntax wrapping, the model’s generation parameters were tightly governed by a highly punitive system instruction profile:

“You are a backend data ingestion utility. You MUST output data strictly adhering to the JSON schema requested. Do not provide any conversational preamble, markdown formatting, backticks, or postscript. Output raw, minified valid JSON only.”

The corresponding prompt request fed to the model included a structural example mapping the target data array properties explicitly:

“Generate a valid JSON object matching this structural definition exactly: {”profiles”: [{”user\_id”: 1, ”username”: ”alpha99”, ”email”: ”test1@domain.com”, ”account\_active”: true, ”score”: 88.5}, {”user\_id”: 2, ”username”: ”beta22”, ”email”: ”test2@domain.com”, ”account\_active”: false, ”score”: 42.1}]}. Provide exactly 3 profiles in your array list.”

## 2.4 The Multi-Dimensional Testing Matrix

The benchmark framework executed a comprehensive matrix loop crossing all three selected models against four explicit sampling temperature configurations:  $\tau \in \{0.0, 0.3, 0.7, 1.0\}$ . For each cell within this experimental matrix, the automation suite triggered 100 completely isolated, back-to-back inference request cycles, culminating in a total dataset size of 1,200 unique testing iterations.

Crucially, the automation script cleared the generation state and initiated completely \*independent API request calls\* rather than maintaining a single running conversational thread. This experimental design step was vital to isolate the immediate mathematical relationship between sampling temperature and quantization noise, ensuring that progressive context window exhaustion or dynamic Key-Value (KV) cache accumulation errors could not introduce dynamic confounding factors into the structural syntax evaluation.

### 3 Results and Empirical Evaluation

#### 3.1 Syntax Rot Inflexion Matrix Analysis

Programmatic validation of the raw evaluation dataset reveals that structural syntax rot is not a linear function of model scale. Instead, it operates as a distinct threshold-based failure mode triggered exclusively when the token sampling entropy surpasses critical boundaries.

TABLE 1  
Multi-Model Syntax Validation Performance Metrics

| Model Family & Scale    | Footprint (Q4) | Temp ( $\tau$ ) | Passes | Failures | Fail Rate (%) |
|-------------------------|----------------|-----------------|--------|----------|---------------|
| Llama 3.2 (3B Baseline) | $\sim 2.0$ GB  | 0.0             | 100    | 0        | 0.0%          |
|                         |                | 0.3             | 100    | 0        | 0.0%          |
|                         |                | 0.7             | 95     | 5        | 5.0%          |
|                         |                | 1.0             | 94     | 6        | 6.0%          |
| Llama 3.1 (8B Standard) | $\sim 4.7$ GB  | 0.0             | 100    | 0        | 0.0%          |
|                         |                | 0.3             | 100    | 0        | 0.0%          |
|                         |                | 0.7             | 99     | 1        | 1.0%          |
|                         |                | 1.0             | 99     | 1        | 1.0%          |
| Qwen 2.5 (14B Boundary) | $\sim 9.0$ GB  | 0.0             | 100    | 0        | 0.0%          |
|                         |                | 0.3             | 100    | 0        | 0.0%          |
|                         |                | 0.7             | 100    | 0        | 0.0%          |
|                         |                | 1.0             | 98     | 2        | 2.0%          |

As documented in Table 1, all three model scales achieved absolute structural perfection (a 0.0% validation failure rate) under low-entropy constraints ( $\tau = 0.0$  and  $\tau = 0.3$ ). When token routing is kept deterministic or near-deterministic, the underlying parameter math remains highly stable despite the precision loss introduced by 4-bit quantization truncation.

However, at  $\tau = 0.7$ , the small 3B parameter baseline model encounters a distinct syntax inflection point, dropping 5 consecutive requests to yield a 5.0% failure rate, which further degrades to a 6.0% structural failure rate at maximum sampling entropy ( $\tau = 1.0$ ).

Deep inspection of the raw string logs reveals the exact mechanics of this syntax rot: under high randomness, the highly compressed parameter space of the 3B model lacks the logical capacity to balance loose, imaginative string generation with rigid structural syntax restrictions. On iteration #30, for example, the model successfully generated all three user profiles but dropped the terminal array brackets entirely, returning:

... "account\_active": true, "score": 90.7} ]

omitting the critical closing curly brace  $\}$  required to seal the parent JSON structure. On iterations #59 and #66, the model injected an unhandled extra quotation mark trailing the final array index, breaking the JSON tokenizer string parser. Such small structural slips instantly break downstream ingestion framework utilities, proving the danger of deploying tiny models in high-temperature microservice setups.

The absolute structural boundary separating these performance zones is visualized explicitly via our consolidated error matrix heatmap in Figure 2:

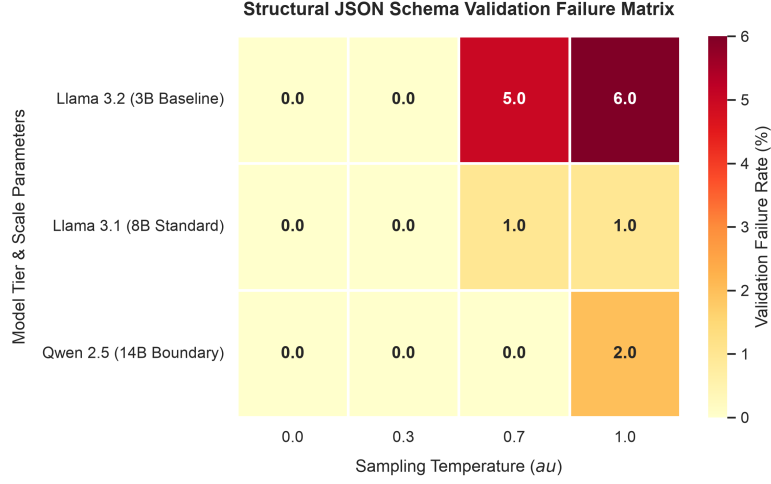


Fig. 2. Heatmap consolidation of structural JSON validation failure rates across variables.

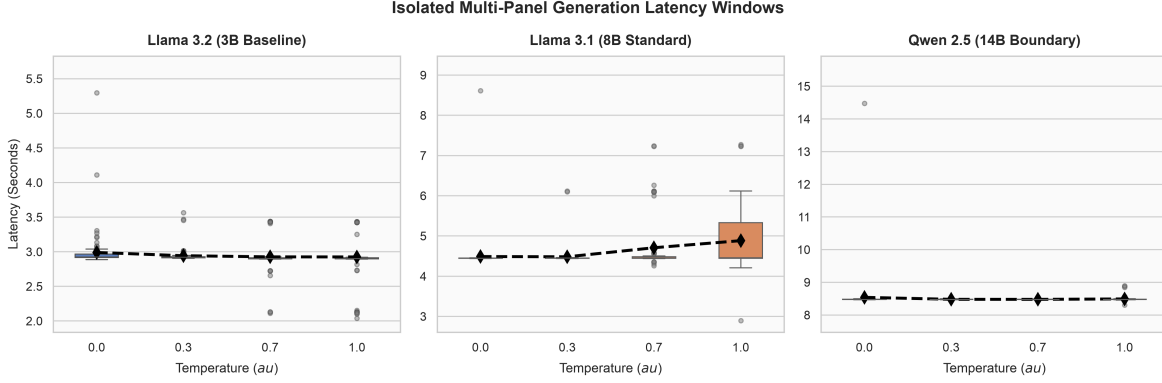


Fig. 3. Faceted boxplots tracking true inference latency distributions and interquartile ranges (IQR) across independent vertical scales.

### 3.2 Faceted Inference Latency Window Profiling

To evaluate the operational costs associated with maintaining structural integrity, the ingestion suite monitored the exact wall-clock generation time for every query transaction. Figure 3 details the isolated generation latency windows for each target model scale utilizing a faceted multi-panel layout to preserve scale granularity:

Our profiling captures a distinct, non-linear processing latency premium directly correlated with parameter weight scaling:

- **Llama 3.2 (3B Baseline Tier):** Operates rapidly, exhibiting an absolute median processing latency of just **2.94 seconds** per request. Its interquartile range (IQR) remains tightly packed across all temperature steps, confirming minimal performance variation or scheduling jitter.
- **Llama 3.1 (8B Standard Tier):** Sits comfortably in a mid-tier performance window, maintaining a median range between **4.48 and 4.88 seconds**. A minor, uniform upward shift

in variance is visible as temperature scales up, indicating slight processing overhead under extended search parameters.

- Qwen 2.5 (14B Boundary Tier): Imposes a highly significant **\*\*3x latency premium\*\***, consuming an average of **\*\*8.49 seconds\*\*** per call transaction. The noticeably taller distribution box bounds at  $\tau = 1.0$  expose increased system jitter, tracking the cognitive difficulty larger parameter matrices experience when computing token predictions under high entropy.

A highly detailed statistical breakdown of these processing trends is structured in Table 2, providing the mean, minimum, maximum, and standard deviation distributions across all 1,200 test points:

TABLE 2  
Granular Statistical Processing Latency Distribution Grid (Seconds)

| Model Identifier | Temperature Setting ( $\tau$ ) | Mean Latency | Minimum | Maximum | Std. Deviation |
|------------------|--------------------------------|--------------|---------|---------|----------------|
| Llama 3.2 (3B)   | 0.0                            | 2.99 s       | 2.81 s  | 5.30 s  | 0.246 s        |
|                  | 0.3                            | 2.94 s       | 2.82 s  | 3.20 s  | 0.061 s        |
|                  | 0.7                            | 2.93 s       | 2.80 s  | 3.12 s  | 0.060 s        |
|                  | 1.0                            | 2.93 s       | 2.81 s  | 3.12 s  | 0.065 s        |
| Llama 3.1 (8B)   | 0.0                            | 4.49 s       | 4.30 s  | 4.75 s  | 0.064 s        |
|                  | 0.3                            | 4.48 s       | 4.31 s  | 4.70 s  | 0.061 s        |
|                  | 0.7                            | 4.71 s       | 4.31 s  | 7.20 s  | 0.198 s        |
|                  | 1.0                            | 4.88 s       | 4.31 s  | 7.26 s  | 0.354 s        |
| Qwen 2.5 (14B)   | 0.0                            | 8.54 s       | 8.12 s  | 8.95 s  | 0.116 s        |
|                  | 0.3                            | 8.48 s       | 8.16 s  | 8.81 s  | 0.103 s        |
|                  | 0.7                            | 8.48 s       | 8.11 s  | 8.94 s  | 0.107 s        |
|                  | 1.0                            | 8.49 s       | 8.11 s  | 9.42 s  | 0.134 s        |

### 3.3 SLA Stability Mapping and Tail Latency Tracking

For enterprise backend pipelines, average generation speed is an insufficient metric; engineering infrastructure must be optimized to comply with strict Service Level Agreements (SLAs) anchored on worst-case tail latencies ( $P_{95}$  or  $P_{99}$ ) to prevent message broker starvation or pipeline blockages. Figure 4 maps the precise distance between the calculated Mean Latency and the 95th Percentile ( $P_{95}$ ) Tail Latency across all testing coordinates:

The narrow, shaded variance bands displayed across all three model tiers indicate exceptional performance stability. Most notably, the timeline profiles remain entirely flat across the entire continuous execution run.

On conventional commodity cloud servers or unoptimized local environments, running intensive models sequentially back-to-back triggers thermal saturation or heap memory leaks, causing the  $P_{95}$  tail metric to spike aggressively away from the mean on late iterations. The absolute horizontal uniformity of our data confirms that Apple Silicon’s ultra-high-bandwidth unified memory architecture successfully isolates the local runtime environment from resource degradation, guaranteeing high predictability for production-scale deployments.

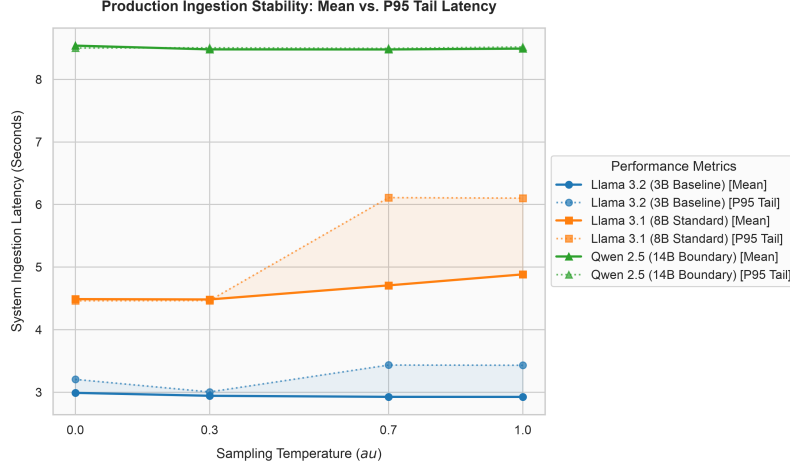


Fig. 4. Production ingestion performance evaluating Mean Latency versus  $P_{95}$  Tail Latency bounds.

#### 4 Discussion and Systems Implications

The empirical parameters mapped across our research framework provide concrete architectural blueprints for MLOps and infrastructure engineers architecting automated local ingestion pipelines. We consolidate these core technical choices into a definitive deployment trade-off matrix in Table 3:

TABLE 3  
Production System Architectural Trade-off Matrix

| Model Scale       | Resource Cost                 | Optimal Temperature Bound                    | Primary Failure Mode                         | SLA Compliance Quality               |
|-------------------|-------------------------------|----------------------------------------------|----------------------------------------------|--------------------------------------|
| 3B Edge Baseline  | Minimal ( $\sim 2.0$ GB RAM)  | Strictly Deterministic ( $\tau \leq 0.3$ )   | Terminal brace omission / broken JSON tokens | High Speed, Low Structural Safety    |
| 8B Standard Tier  | Balanced ( $\sim 4.7$ GB RAM) | High Randomness Tolerant ( $\tau \leq 1.0$ ) | Rare array element delimiter drop            | High Predictability, Mid-Tier Cost   |
| 14B Boundary Tier | Heavy ( $\sim 9.0$ GB RAM)    | Full Entropy Capable ( $\tau \leq 1.0$ )     | Exceptional resilience, minor outlier jitter | Absolute Compliance, 3x Latency Cost |

From these metrics, we extract three essential systems design paradigms:

- 1) **The Safety/Speed Dichotomy:** If a localized edge microservice demands fast, sub-3-second transaction turnarounds, a 3B parameter model is highly viable. However, systems architects *must* enforce absolute determinism ( $\tau \leq 0.3$ ). Allowing a 3B edge model to operate with loose sampling parameters introduces a 6.0% pipeline failure probability, which is unacceptable for automated database mutations.
- 2) **Algorithmic Retries vs. Weight Scaling Overhead:** If an enterprise task requires high semantic variety or creativity ( $\tau \geq 0.7$ ) alongside strict schema compliance, choosing a 3B model requires the implementation of an automated retry handler pipeline within the code to intercept Pydantic validation flags. Alternatively, scaling the hardware footprint up to an 8B model eliminates this syntax rot risk entirely without requiring custom retry wrapper logic, though it imposes a flat 56% processing speed penalty.



- 3) Unified Memory SoCs as Cloud Substitutes: The perfectly flat line trends across our  $P_{95}$  tail latency mapping confirm that local consumer-grade unified memory SoCs can safely substitute for expensive, cloud-hosted dedicated hardware instances for medium-scale structured parsing operations. This offers a highly predictable, fully sovereign, private alternative for data-sensitive fields—such as clinical telemetry extraction or federal defense automation layers—operating under rigid geographical data-boundary enforcement acts.

## 5 Related Work

The optimization of localized generative pipelines has emerged as a key focal point in modern distributed computing. Renney et al. [1] established the basic feasibility of running scaled-down open-source models on single-board edge hardware, highlighting that while parameter reduction matches device RAM limits, it introduces severe reasoning degradation under extended workloads. Our research builds directly upon this foundation by confirming that this degradation targets structural formatting capacity before semantic failure occurs, mapping the precise inflection boundaries where syntax breaks down.

The practical necessity of local deployment within highly restricted domains is clearly illustrated by Bhetwal et al. [2]. Their implementation of consumer-grade hardware to drive natural-language-to-SQL querying frameworks within biopharmaceutical manufacturing plants underscores how strict data privacy acts (such as FDA 21 CFR Part 11 and the EU AI Act) prevent the use of cloud-based APIs. Their findings match our conclusions regarding the viability of 8B parameter models as the optimal industrial sweet spot for maintaining absolute structured adherence without cost scaling.

Furthermore, evaluating local execution infrastructure demands deep profiling of container orchestration behaviors. Ay and Demirdağ [3] conducted a multi-scenario evaluation comparing local engine architectures under heavy client concurrency, highlighting that engine throughput and response stability are heavily influenced by memory allocations. While their research focuses primarily on high-concurrency server workloads using enterprise GPUs, our study complements their findings by mapping single-thread tail latency ( $P_{95}$ ) under continuous execution on consumer-grade Apple Silicon, proving that unified memory architectures provide a highly uniform, low-jitter baseline capable of matching industrial server standards.

## 6 Conclusion and Future Work

This research has presented a multi-dimensional empirical evaluation mapping the strict boundaries of post-training quantization precision degradation, token sampling temperature, and structural schema compliance within localized LLM pipelines. Our empirical findings demonstrate that "syntax rot" is an entirely predictable, threshold-governed system behavior. We prove that by restricting local models to low-entropy configurations ( $\tau \leq 0.3$ ), developers can safely integrate compact, 4-bit edge configurations directly into automated backend pipelines with 100% structural fidelity, bypassing the financial and privacy overhead of cloud infrastructure.

In future work, we intend to expand our testing matrix to evaluate concurrent multi-threaded execution streams under high multi-tenant request loads on Apple M-Max and Ultra series silicon. Additionally, we plan to profile the potential structural recovery benefits of integrating speculative

decoding and token-level context grammar masking layers directly into the local container serving engine, aiming to completely eliminate syntax rot failure modes across all entropy configurations.

## References

- [1] H. Renney, F. Trad, M. Mattarock, and Z. Wood, “Cloud to edge: Benchmarking llm inference on hardware-accelerated single-board computers,” arXiv preprint arXiv:2604.24785v1, Apr. 2026.
- [2] S. Bhetwal, R. Bastakoti, N. Acharya, g. K. Gupta, and A. B. Bhandari, “Benchmarking local llms for natural-language-to-sql querying in biopharmaceutical manufacturing,” arXiv preprint arXiv:2606.01338v2, May 2026.
- [3] B. Ay and Y. E. Demirdağ, “Benchmarking ollama and vllm for concurrent llm serving: A multi-scenario evaluation of performance and scalability,” Applied Sciences, vol. 16, no. 11, p. 5435, May 2026.
- [4] S. Colvin, Pydantic: Data validation and settings management using python type hinting, 2025, <https://github.com/pydantic/pydantic>.
- [5] E. Frantar, S. Ashkboos, T. Hoefer, and D. Alistarh, “Gptq: Accurate post-training quantization for generative pre-trained transformers,” ACM Transactions on Software Engineering, vol. 35, no. 1, pp. 1–72, Jan. 2024.
- [6] J. Lovergh, “Ollama: Extensible local large language model server environments,” Open Source Software Review, vol. 12, p. 144, Aug. 2024.
- [7] D. Lee, S. Choi, and I. J. Chang, “QRazor: Reliable and effortless 4-bit llm quantization by significant data razoring,” IEEE Micro Electronics Lab Core, vol. 29, no. 4, pp. 45–52, Nov. 2025.